

webkit.org. It is open source and provides the ability to render Web pages embedded in the JavaFX application. It supports HTML5, CSS, Java-Script, DOM and SVG.

Major JavaFX features

The main features of JavaFX are listed below.

- **Availability:** The JavaFX APIs are available as part of the Java Runtime Environment (JRE) and Java Distribution Kit (JDK) distribution, and do not require any special configuration to be used in applications. These APIs can directly call other Java APIs used in the application or business logic implementation layer.
- **Scene graph model:** JavaFX is based on a scene graph model, where UI elements such as buttons, labels, text fields, etc., can be defined as hierarchical structures and transforms can be applied to position elements. Scene graph allows UI creation with much less effort than Swing and gives high performance, using the hardware accelerated graphics pipeline especially crucial for 3D animation.
- **CSS styling of UI controls:** JavaFX provides all the UI controls needed to design a full-featured application. As of JavaFX 8, all the controls can be styled using CSS, similar to HTML page styling. The root node can be styled to give a new look to the entire frame without changing any Java code.
- **Web View:** This is a Web component which allows you to embed Web pages in the JavaFX application. It is based on the open source browser engine, Webkit, and supports the latest HTML5, CSS, JavaScript and additional features such as Web socket, Web workers, Web fonts and printing capabilities.
- **Swing interoperability:** JavaFX introduces the SwingNode class, using which Swing contents can be embedded in JavaFX. In the same way, Swing applications can embed JavaFX using the JFXPanel.
- **Charts:** JavaFX has a rich package that includes several chart types: pie charts, bar charts, line charts, bubble charts, scatter charts, area charts, stacked area charts, stacked bar charts, etc.
- **3D graphics:** JavaFX has introduced Shape3D APIs, which support Box, Cylinder, Mesh View and Sphere sub-classes; SubScene, Material, PickResult, LightBase (AmbientLight and PointLight) sub-classes, etc. The Camera API class has also been updated in this release.

JavaFX versus Swing versus HTML5/CSS3

Swing was a UI technology developed in the 90s. It does not offer support for touch, animation and data binding. It has no support for WebView and media as well. JavaFX addresses all these shortcomings. Its differentiating feature, scene graph, is at the core of the API. UI widgets are thus like any other node in the scene graph and can be animated, transformed, filtered and can leverage customised event handling.

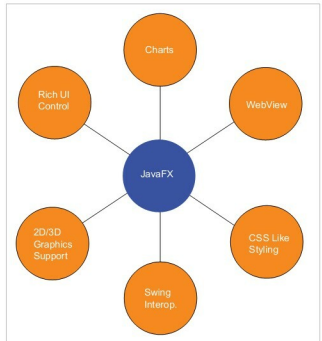


Figure 2: JavaFX features

JavaFX content can be embedded in Swing applications using JFXPanel. All user input events (key, mouse, and focus events) are transparently forwarded to the JavaFX scene. Similarly, a customised Swing component can be embedded in JavaFX with the SwingNode control. This option allows users to keep Swing working but still migrate to the JavaFX world.

A combination of HTML5 and CSS3 is a good platform for cross-platform UI development. This is especially true with the new HTML5 and Apache Caradova, where the entire application logic can be written in JavaScript, avoiding any server side development needed in JSP, JSF, etc. This kind of development falls under native HTML5 application development, which is run with the help of WebView, using an embedded browser. But when it comes to large data handling and integration of applications with the server, JavaFX has an advantage. Server side communication in JavaFX is supported by Websocket, REST or JSON processing, whereas HTML5/CSS3 depends on native support of REST and Websocket. JavaFX applications integrate with server side Java comparatively more easily, needing the same set of Java skills.

HTML5 development challenges include the inability to recover from fatal errors, different approaches to defining classes, and no IDE coding support and threading – Web workers cannot update the document object model (DOM). JavaFX challenges include mobile support and interaction with hardware. HTML5, with features such as 2D Canvas, webgl, CSS3 (WebFonts, transformations and CSS animation), remains a good option for hybrid mobile apps and native mobile application development.